

BINARY SEARCH

```
Sorted array?
|
YES
|
Exact element?
|
-> Normal Binary Search

Duplicates?
|
-> First/Last Occurrence

First >= target?
|
-> Lower Bound

First > target?
|
-> Upper Bound

Rotated?
|
-> Rotated Binary Search

Minimum/Maximum Answer?
|
-> Binary Search on Answer

Peak/Mountain?
|
-> Peak Element

Matrix?
|
-> Matrix Binary Search
```



Binary Search Pattern Sheet

Pattern 1: Search in Sorted Array

When?

- Array is sorted.
- Need exact element.

Template

```
int binarySearch(vector<int>& nums, int target) {
    int low = 0, high = nums.size() - 1;

    while(low <= high) {
        int mid = low + (high - low) / 2;

        if(nums[mid] == target)
            return mid;
        else if(nums[mid] < target)
            low = mid + 1;
        else
```

```

        high = mid - 1;
    }

    return -1;
}

```

Questions

- Binary Search

```

public:
    int search(vector<int>& nums, int target) {
        int n=nums.size();
        int low=0;
        int high=n-1;
        while(low<=high){
            int mid=(high+low)/2;
            if(nums[mid]==target){
                return mid;
            }
            else if(nums[mid]>target){
                high=mid-1;
            }
            else{
                low=mid+1;
            }
        }
        return -1;
    }
};

```

- Search Insert Position

```

class Solution {
public:
    int searchInsert(vector<int>& nums, int target) {
        int n=nums.size();
        int i=0;
        int j=n-1;
        int ans=n;
        while(i<=j){
            int mid=(j+i)/2;
            if(nums[mid]==target){
                return mid;
            }
            else if(nums[mid]>target){
                ans=mid;
                j=mid-1;
            }
            else{
                i=mid+1;
            }
        }
        return ans;
    }
};

```

- Guess Number Higher or Lower

```

public:
    int guessNumber(int n) {
        int l=0;
        int h=n;
        while(l<=h){
            int mid=(l+h)/2;
            int result=guess(mid);
            if(result==0){
                return mid;
            }
            else if(result==-1){
                h=mid-1;
            }
            else{
                l=mid+1;
            }
        }
        return -1;
    }
};
// -1 → Guess is HIGH
//     Move LEFT

// 1 → Guess is LOW
//     Move RIGHT

```

Pattern 2: First Occurrence

When?

Duplicates exist.

Example:

```

1 2 2 2 3 4
    ↑
First 2

```

Template

```

int firstOccurrence(vector<int>& nums, int target) {
    int low = 0, high = nums.size()-1;
    int ans = -1;

    while(low <= high) {
        int mid = low + (high-low)/2;

        if(nums[mid] == target) {
            ans = mid;
            high = mid - 1;
        }
        else if(nums[mid] < target)
            low = mid + 1;
        else
            high = mid - 1;
    }
}

```

```
    }  
    return ans;  
}
```

Questions

- First Occurrence
 - Find First and Last Position
-

Pattern 3: Last Occurrence

```
int lastOccurrence(vector<int>& nums, int target) {  
    int low = 0, high = nums.size()-1;  
    int ans = -1;  
  
    while(low <= high) {  
        int mid = low + (high-low)/2;  
  
        if(nums[mid] == target) {  
            ans = mid;  
            low = mid + 1;  
        }  
        else if(nums[mid] < target)  
            low = mid + 1;  
        else  
            high = mid - 1;  
    }  
  
    return ans;  
}
```

Questions

- Find First and Last Position
-

Pattern 4: Lower Bound

Find first element $\geq$ target

Example

```
1 3 5 7 9  
target = 4
```

Answer = 5 (index 2)

Template

```

int lowerBound(vector<int>& nums, int target) {
    int low = 0, high = nums.size()-1;
    int ans = nums.size();

    while(low <= high) {
        int mid = low + (high-low)/2;

        if(nums[mid] >= target) {
            ans = mid;
            high = mid - 1;
        }
        else
            low = mid + 1;
    }

    return ans;
}

```

Questions

- Search Insert Position

```

//.....
int searchInsert(vector<int>& nums, int target) {
    int low=0;
    int high=nums.size()-1;
    int ans=nums.size();
    while(low<=high){
        int mid=(low+high)/2;
        if(nums[mid]==target){
            return mid;
        }
        else if(nums[mid]>target){
            ans=mid;
            high=mid-1;
        }
        else{
            low=mid+1;
        }
    }
    return ans;
}
;

```

- Lower Bound

Pattern 5: Upper Bound

First element > target

Example

1 2 2 2 5

target = 2

Answer = index 4

Template

```

int upperBound(vector<int>& nums, int target) {
    int low = 0, high = nums.size()-1;
    int ans = nums.size();

    while(low <= high) {
        int mid = low + (high-low)/2;

        if(nums[mid] > target) {
            ans = mid;
            high = mid - 1;
        }
        else
            low = mid + 1;
    }

    return ans;
}

```

Pattern 6: Binary Search on Answer

Most important interview pattern.

Clue

Question asks:

- Minimum possible ?
- Maximum possible ?
- Smallest ?
- Largest ?

Template

```

int low = minValue;
int high = maxValue;

while(low <= high) {
    int mid = low + (high-low)/2;

    if(isPossible(mid)) {
        ans = mid;
        high = mid - 1;
    }
    else {
        low = mid + 1;
    }
}

```

Tree

Find minimum answer

```

|
mid works?

```

```

    /      \
   yes     no
high=mid-1 low=mid+1

```

Questions

- Koko Eating Bananas

```

2 public:
3 long long checkk(int mid,vector<int>& piles){
4     long long count=0;
5     for(auto x:piles){
6         count += ceil((double)x / mid);
7     }
8     return count;
9 }
10 int minEatingSpeed(vector<int>& piles, int h) {
11     //here we have to take l=1 and h=max elemnt of piles;
12     int low=1;
13     int high=*max_element(piles.begin(),piles.end());
14     int ans=0;
15     while(low<=high){
16         int mid=(low+high)/2;
17         long long check=checkk(mid,piles);
18         if(check<=h){
19             ans=mid;
20             high=mid-1;
21         }
22         else{
23             low=mid+1;
24         }
25     }
26
27     return ans;
28 }
29 }:

```

- Capacity To Ship Packages
- Split Array Largest Sum –in the last
- Min Days To Make Bouquets
- Aggressive Cows –
- Painter Partition –in the last

Pattern 7: Rotated Sorted Array

Clue

Array sorted + rotated

Example

4 5 6 7 0 1 2

Template

```

while(low <= high) {
    int mid = low + (high-low)/2;

```

```

    if(nums[mid] == target)
        return mid;

    if(nums[low] <= nums[mid]) {
        if(target >= nums[low] &&
           target < nums[mid])
            high = mid - 1;
        else
            low = mid + 1;
    }
    else {
        if(target > nums[mid] &&
           target <= nums[high])
            low = mid + 1;
        else
            high = mid - 1;
    }
}

```

Questions

- Search in Rotated Sorted Array -done
- Search in Rotated Sorted Array II

Pattern 8: Find Peak Element

Clue

Need local maximum.

```

while(low < high) {
    int mid = low + (high-low)/2;

    if(nums[mid] < nums[mid+1])
        low = mid + 1;
    else
        high = mid;
}

```

Questions

- Find Peak Element


```

1 class Solution {
2 public:
3     int findPeakElement(vector<int>& arr) {
4         int n=arr.size();
5         if(n==1) return 0;
6         if(arr[0]>arr[1]) return 0;
7         if(arr[n-1]>arr[n-2]) return n-1;
8         int low=1;
9         int high=arr.size()-2;
10        while(low<=high){
11            int mid=(low+high)/2;
12            if(arr[mid]>arr[mid-1]&&arr[mid]>arr[mid+1]){
13                return mid;
14            }
15            else if(arr[mid]>arr[mid-1]){
16                low=mid+1;
17            }
18            else{
19                high=mid-1;
20            }
21        }
22        return -1;
23    }
24 }

```

- Peak Index in Mountain Array

```

1 class Solution {
2 public:
3     int findPeakElement(vector<int>& arr) {
4         int n=arr.size();
5         if(n==1) return 0;
6         if(arr[0]>arr[1]) return 0;
7         if(arr[n-1]>arr[n-2]) return n-1;
8         int low=1;
9         int high=arr.size()-2;
10        while(low<=high){
11            int mid=(low+high)/2;
12            if(arr[mid]>arr[mid-1]&&arr[mid]>arr[mid+1]){
13                return mid;
14            }
15            else if(arr[mid]>arr[mid-1]){
16                low=mid+1;
17            }
18            else{
19                high=mid-1;
20            }
21        }
22        return -1;
23    }
24 }

```

Pattern 9: Infinite Sorted Array

Step 1

Expand range

```

while(target > nums[high]) {
    low = high;
    high *= 2;
}

```

Step 2

Apply Binary Search.

Pattern 10: Matrix Binary Search

Matrix Sorted

```
int low = 0;
int high = m*n-1;

while(low <= high) {
    int mid = low + (high-low)/2;

    int row = mid / n;
    int col = mid % n;

    if(matrix[row][col] == target)
        return true;

    if(matrix[row][col] < target)
        low = mid + 1;
    else
        high = mid - 1;
}
```

Questions

- Search a 2D Matrix

Pattern to remember

For any matrix:

```
row = index / number_of_columns;
col = index % number_of_columns;
```

Find Minimum in Rotated Sorted Array

```
public:
    int findMin(vector<int>& nums) {
        int n=nums.size();
        int low=0;
        int high=nums.size()-1;
        int ans=INT_MAX;
        while(low<=high){
            int mid=(low+high)/2;
            if(nums[low]<=nums[mid]){
                ans=min(ans,nums[low]);
                low=mid+1;
            }
            else{
                ans=min(ans,nums[mid]);
                high=mid-1;
            }
        }
        return ans;
    }
};
```

Must-Do Binary Search Questions (Top 20)

1. Binary Search -done
2. Search Insert Position -done
3. Find First and Last Position -done
4. Lower Bound -done
5. Upper Bound -done
6. Sqrt(x) -done
7. Search in Rotated Sorted Array -done
8. Search in Rotated Sorted Array II
9. Find Minimum in Rotated Sorted Array -done
10. Single Element in Sorted Array –done but by bit manipulation
11. Find Peak Element -done
12. Peak Index in Mountain Array -done
13. Search a 2D Matrix -done
14. Koko Eating Bananas -done
15. Capacity To Ship Packages Within D Days
16. Min Days to Make Bouquets
17. Smallest Divisor Given Threshold -done
18. Aggressive Cows
19. Split Array Largest Sum -done
20. Median of Two Sorted Arrays -done
21. Allocate books -done
22. The painters partition -done

Sqrt(x):

```
public:
    int mySqrt(int x) {
        if(x<=1) return x;
        int low=0;
        int high=x/2;
        int ans=0;
        while(low<=high){
            int mid=(low+high)/2;
            long long sq=1LL* mid*mid;
            if(sq==x){
                return mid;
            }
            else if(sq<x){
                ans=mid;
                low=mid+1;
            }
            else{
                high=mid-1;
            }
        }
        return ans;
    }
};
```

Allocate Minimum Pages / The Painter's Partition / Split Array Largest Sum

```
3 bool check(vector<int>& nums,int k,int allowed){
4     int pages=0;
5     int stu=1;
6     for(int i=0;i<nums.size();i++){
7         if(nums[i]>allowed){
8             return false;
9         }
10        if(pages+nums[i]<=allowed){
11            pages+=nums[i];
12        }
13        else{
14            stu++;
15            pages=nums[i];
16        }
17    }
18    if(stu>k) return false;
19    return true;
20 }
21 int splitArray(vector<int>& nums, int k) {
22     int sum=0;
23     int n=nums.size();
24     int low=0;
25     int high=0;
26     for(auto x:nums){
27         sum+=x;
28     }
29     high=sum;
30     int ans=-1;
31     while(low<=high){
32         int mid=(low+high)/2;
33         int checkk=check(nums,k,mid);
34         if(checkk==true){
35             ans=mid;
36             high=mid-1;
37         }
38         else{
39             low=mid+1;
40         }
41     }
42     return ans;
43 }
```

Find the Smallest Divisor Given a Threshold-

isme hamlog ko ek positive divisor find krna h joh sabse minimum ho jisse hamlog array ke sabhi no. ko divide kre aur joh divide krne pe aae usko sabko plus kre aur sum vali value agr threshold value se kam ho toh voh ans ho skta h

```

1 class Solution {
2 public:
3 bool isValid(vector<int>& nums, int threshold, int mid){
4     int count=0;
5     for(auto x:nums){
6         count+=ceil((double)x/mid);
7     }
8     return count<=threshold;
9 }
10 int smallestDivisor(vector<int>& nums, int threshold) {
11     int n=nums.size();
12     int low=1;
13     int high=*max_element(nums.begin(),nums.end());
14     int ans=high;
15     while(low<=high){
16         int mid=(low+high)/2;
17         if(isValid(nums,threshold,mid)){
18             ans=mid;
19             high=mid-1;
20         }
21         else{
22             low=mid+1;
23         }
24     }
25     return ans;
26 }
27 }
28 };

```

What are they asking?

You have an array:

```

nums = [1,2,5,9]
threshold = 6

```

You need to choose a **positive divisor**.

For each element:

```
ceil(nums[i] / divisor)
```

and then add all the results.

The goal is:

✓ Find the **smallest divisor** such that the total sum is $\leq$ **threshold**.

Example

Suppose divisor = 1

```

ceil(1/1) = 1
ceil(2/1) = 2
ceil(5/1) = 5
ceil(9/1) = 9

```

Sum: $1 + 2 + 5 + 9 = 17$ But: $17 > 6$ Not valid.

Minimum Number of Days to Make m Bouquets

```

2 public:
3 bool check(int mid,vector<int>& bloomDay,int m,int k){
4     int n=bloomDay.size();
5     int count=0;
6     int num=0;
7     for(int i=0;i<n;i++){
8         if(bloomDay[i]<=mid){
9             count++;
10        }
11        else{
12            num+=(count/k);
13            count=0;
14        }
15    }
16    num+=(count/k);
17    return num>=m;
18 }
19 int minDays(vector<int>& bloomDay, int m, int k) {
20     int n=bloomDay.size();
21     int maxi=*max_element(bloomDay.begin(),bloomDay.end());
22     int mini=*min_element(bloomDay.begin(),bloomDay.end());
23     long long val = m * 111 * k * 111;
24     if(val>n) return -1;
25     int low=mini;
26     int high=maxi;
27     while(low<=high){
28         int mid=(low+high)/2;
29         if(check(mid,bloomDay,m,k)){
30             high=mid-1;
31         }
32         else{
33             low=mid+1;
34         }
35     }
36     return low;
37 }
38 };

```

Capacity To Ship Packages Within D Days

```

1 class Solution {
2 public:
3 bool checkk(vector<int>& nums,int days,int mid){
4     int sum=0;
5     int count=1;
6     for(int i=0;i<nums.size();i++){
7         if(sum+nums[i]<=mid){
8             sum+=nums[i];
9         }
10        else{
11            count++;
12            sum=nums[i];
13        }
14    }
15    return count<=days;
16 }
17 int shipWithinDays(vector<int>& weights, int days) {
18     int low=*max_element(weights.begin(),weights.end());
19     int high=accumulate(weights.begin(),weights.end(),0);
20     int ans=0;
21     while(low<=high){
22         int mid=(low+high)/2;
23         int check=checkk(weights,days,mid);
24         if(check==true){
25             ans=mid;
26             high=mid-1;
27         }
28         else{
29             low=mid+1;
30         }
31     }
32     return ans;
33 }
34 };

```